

Digital Design Diploma

100 Interview Questions

Coverage: Verilog HDL, TCL, SpyGlass Lint/CDC
Synthesis (DC), STA (PrimeTime)
Power Analysis, CDC Design
Formal Verification (Formality)
DFT, GLS, Physical Design (PnR)

Difficulty: 70% Advanced 30% Moderate

Formats: Conceptual, Scenario-Based, Code Review,
Mathematical, True/False, Fill-in-the-Blank

September 26, 2026

Contents

1	Verilog HDL & Digital Architecture (12 Questions)	2
2	TCL Scripting for EDA (8 Questions)	7
3	SpyGlass Lint & CDC Static Verification (10 Questions)	10
4	ASIC Synthesis — Synopsys Design Compiler (12 Questions)	14
5	Static Timing Analysis — Synopsys PrimeTime (12 Questions)	19
6	Clock Domain Crossing (CDC) Design (10 Questions)	23
7	Power Analysis & Clock Gating (5 Questions)	27
8	Formal Verification — Logic Equivalence Checking (8 Questions)	29
9	Design for Testability (DFT) & Scan Insertion (10 Questions)	32
10	Gate-Level Simulation (GLS) & Back-Annotation (5 Questions)	35
11	Physical Design — Place & Route (8 Questions)	37
	Summary Statistics	41
	Interview Tips & Formula Reference	42

1 Verilog HDL & Digital Architecture (12 Questions)

Q1 **ADVANCED** — Blocking vs. Non-Blocking Race Condition *Code Review / Scenario*

Consider the following Verilog snippet:

```
1  always @(posedge clk) begin
2      a = b;
3      b = a;
4  end
```

- What is the result after the clock edge — does a swap occur? Explain why.
- Rewrite the block using non-blocking assignments to achieve the intended swap behavior.
- What is the underlying simulator execution model that makes non-blocking assignments safe for sequential logic?

Expected Answer

With blocking =, a gets the old value of b, then b gets the **new** value of a (which already holds old b) — no swap occurs, both end up equal.

Non-blocking <= schedules all RHS evaluations first (simultaneously during the “evaluate” phase), then performs all LHS assignments (in the “update” phase) — producing a true swap:

```
1  always @(posedge clk) begin
2      a <= b;
3      b <= a;
4  end
```

The scheduler separates “evaluate” from “update” phases, making <= safe for concurrent sequential register transfers.

Q2 **ADVANCED** — Latch Inference Diagnosis

Code Review

```
1  always @(*) begin
2      case (sel)
3          2'b00: out = a;
4          2'b01: out = b;
5          2'b10: out = c;
6      endcase
7  end
```

- Will this code infer a latch? Why or why not?
- Provide two correct methods to fix this without changing the functional intent.
- What SpyGlass rule ID is triggered by this violation, and what severity does it carry?

Expected Answer

Yes — `sel = 2'b11` has no assignment, so `out` must retain its previous value $\Rightarrow$ D-Latch is inferred.

Fix 1 — Add default case: `default: out = 'x;`

Fix 2 — Assign default at top of `always` block:

```

1  always @(*) begin
2      out = 'x;    // default kills all latches
3      case (sel)
4          2'b00: out = a;
5          2'b01: out = b;
6          2'b10: out = c;
7      endcase
8  end

```

SpyGlass rule: InferLatch / W188, severity **Error**.

Q3 MODERATE — FSM Architecture Styles*Conceptual*

Compare **Mealy vs. Moore** FSM output styles and explain the **3 advantages** of using the 3-process registered-output FSM style over the classic 2-process Mealy style in ASIC design.

Expected Answer

Moore: outputs depend only on current state. **Mealy:** outputs depend on both state and inputs.

3-process advantages:

1. **Glitch-free outputs** — registered outputs eliminate combinational glitches on asynchronous signals.
2. **Setup-friendly** — the output register adds one cycle latency but dramatically improves timing closure.
3. **DFT-ready** — registered outputs are standard D flip-flops, fully accessible via scan chains.

Q4 ADVANCED — LFSR & CRC Design*Conceptual + Mathematical*

- a) For the CRC-8 polynomial $x^8 + x^2 + x + 1$, identify the feedback tap positions in the LFSR.
- b) What is the maximal-length sequence period for an 8-bit LFSR?
- c) Why is an all-zeros state forbidden in an LFSR, and how is it avoided in hardware?

Expected Answer

- a) Taps at positions 8, 2, 1, 0 $\Rightarrow$ XOR feedback from bits [7], [1], [0] back to the bit [7] input.

- b) Period = $2^8 - 1 = 255$.
- c) All-zeros locks in a zero-only loop forever. Avoided by loading a non-zero seed at reset or adding an extra gate (e.g., OR of all states) to inject a 1.

Q5 **ADVANCED** — ALU Flags & Overflow Detection*Conceptual*

For a 16-bit signed ALU performing subtraction ($A - B$):

- a) Write the Boolean expression for detecting **signed overflow**.
- b) When does the **Carry** flag differ from the **Overflow** flag in a subtraction?
- c) What is the difference between arithmetic right shift and logical right shift for signed numbers?

Expected Answer

- a) Overflow = $(A[15] \oplus B[15]) \& (A[15] \oplus \text{result}[15])$ — signs of operands differ AND result sign differs from A.
- b) Carry indicates unsigned borrow/overflow; Overflow signals signed overflow. They are independent and can each be 0 or 1 simultaneously.
- c) Arithmetic right shift replicates (sign-extends) the MSB; logical right shift fills with 0.

Q6 **ADVANCED** — Parameterized Register File*Code Review*

```

1 module RegFile #(parameter DEPTH=8, parameter WIDTH=8) (
2     input  clk, wr_en,
3     input  [$clog2(DEPTH)-1:0] wr_addr, rd_addr,
4     input  [WIDTH-1:0] wr_data,
5     output reg [WIDTH-1:0] rd_data
6 );
7     reg [WIDTH-1:0] mem [0:DEPTH-1];
8     always @(posedge clk) begin
9         if (wr_en) mem[wr_addr] <= wr_data;
10        rd_data <= mem[rd_addr];
11    end
12 endmodule

```

- a) What is the read behavior — synchronous or asynchronous? What timing implication does this have?
- b) If $\text{wr_addr} == \text{rd_addr}$ and $\text{wr_en} == 1$ simultaneously, what value does rd_data capture (new or old)?
- c) How would you implement “read-first” vs. “write-first” behavior?

Expected Answer

- a) **Synchronous** read — rd_data is registered, adding one cycle read latency. STA sees this as a Reg-to-Reg path through the memory array.

b) **Old value** — because both assignments use `<=`, the read captures the pre-edge array content.

c) **Write-first:**

```
1  if (wr_en && wr_addr == rd_addr)
2      rd_data <= wr_data;
3  else
4      rd_data <= mem[rd_addr];
```

Q7 **MODERATE** — Testbench Self-Checking

Conceptual

What is the difference between a **directed testbench** and a **self-checking testbench**? List three elements a professional self-checking testbench must contain.

Expected Answer

A directed TB manually applies vectors; a self-checking TB automatically compares DUT outputs against expected values using assertions.

Three mandatory elements:

1. **Clock generator** with a precise, parameterised period.
2. **Driver tasks** that systematically apply stimulus.
3. **Checker/scoreboard** with `$display` pass/fail messages, an `err_count` register, and a final `$finish`.

Q8 **ADVANCED** — Sensitivity List Pitfalls

Code Review

```
1  always @(a or b) begin
2      c = a & b;
3      d = c | e;
4  end
```

- a) What SpyGlass rule is violated? What is the fix?
- b) What simulation bug occurs when `e` changes but `a` and `b` do not?
- c) Does the sensitivity list affect the synthesised gate-level netlist? Explain.

Expected Answer

- a) **W18** — incomplete sensitivity list. Fix: `always (*)` or SystemVerilog `always_comb`.
- b) `d` will not update when `e` changes alone — RTL simulation diverges from actual hardware behaviour.
- c) **No** — synthesis ignores the sensitivity list entirely and infers gates from the dataflow. The mismatch causes Sim/Synth divergence only in simulation.

Q9 **MODERATE** — full_case / parallel_case Dangers **Conceptual**

- a) What does `// synthesis parallel_case` instruct the synthesis tool to do?
- b) Why does SpyGlass flag this with rule W398?
- c) What is the correct, synthesisable alternative?

Expected Answer

- a) Tells synthesis each case arm is mutually exclusive $\Rightarrow$ synthesises a priority MUX as a flat parallel MUX (saves area/timing).
- b) Simulation still evaluates case with priority, so simulator and synthesis tool produce different logic $\Rightarrow$ functional mismatch.
- c) Explicitly code all non-overlapping conditions or add `default:` so both tools see identical logic without relying on pragmas.

Q10 **ADVANCED** — assign vs. always vs. Procedural Timing *Fill-in-the-Blank*

Complete the table:

Statement	Evaluated when...	Models...	Synthesises to...
<code>assign y = a & b;</code>	(1)	(2)	(3)
<code>always @(*) with =</code>	(4)	(5)	(6)
<code>always @(posedge clk) with <=</code>	(7)	(8)	(9)

Expected Answer

- (1) Any time a or b changes; (2) Continuous wire-level logic; (3) AND gate;
 (4) Any input changes; (5) Combinational logic; (6) Gates / MUX;
 (7) Active clock edge; (8) Sequential register transfer; (9) D flip-flop.

Q11 **ADVANCED** — Multi-Bit Counter with Terminal Flags **Design**

Design a **parameterised synchronous up/down counter** that:

- Has synchronous active-high load.
- Asserts **High** flag when at max value ($2^{\text{WIDTH}} - 1$).
- Asserts **Low** flag when at min value (0).
- Uses non-blocking assignments throughout.

Write the core **always** block and the flag **assign** statements.

Expected Answer

```

1 parameter WIDTH = 8;
2 reg [WIDTH-1:0] Counter;
3 input Up, Down, Load, CLK, RST_N;

```

```

4  input [WIDTH-1:0] IN;
5  output High, Low;
6
7  always @(posedge CLK or negedge RST_N) begin
8      if (!RST_N) Counter <= {WIDTH{1'b0}};
9      else if (Load) Counter <= IN;
10     else if (Up) Counter <= Counter + 1;
11     else if (Down) Counter <= Counter - 1;
12 end
13
14 assign High = (Counter == {WIDTH{1'b1}});
15 assign Low = (Counter == {WIDTH{1'b0}});

```

Q12 **MODERATE** — Synthesisable vs. Non-Synthesisable Constructs *True/False*

State whether each construct is synthesisable:

Construct	Synthesisable?
initial begin ...end	<u>(1)</u>
#10 a = 1;	<u>(2)</u>
\$display("hello")	<u>(3)</u>
always @(*)	<u>(4)</u>
integer i; for (i=0; i<4; i=i+1)	<u>(5)</u>

Expected Answer

(1) **No** — simulation-only initialisation; (2) **No** — absolute time delay; (3) **No** — system task; (4) **Yes**; (5) **Yes** — if loop bounds are constants (statically unrolled by synthesis).

2 TCL Scripting for EDA (8 Questions)

Q13 **MODERATE** — TCL Substitution Rules

Scenario

Predict the output of each line:

```

1  set x 10
2  set y "x is $x"
3  set z {x is $x}
4  puts $y
5  puts $z
6  puts "Double: [expr $x * 2]"

```

Expected Answer

x is 10 / x is \$x / Double: 20

Double quotes perform variable and command substitution; curly braces suppress all substitution.

Q14 **ADVANCED** — EDA Collection Iteration**Scenario**

In Design Compiler, which option is correct for iterating over input ports?

```

1  # Option A
2  foreach port [get_ports *] { puts $port }
3
4  # Option B
5  foreach_in_collection port [get_ports -filter "direction
   == in"] {
6      puts [get_attribute $port name]
7  }
```

Expected Answer

Option B is correct. `get_ports` returns a DC **collection object**, not a Tcl list; `foreach` cannot iterate over it. `foreach_in_collection` is the correct EDA command, and `get_attribute` extracts the name string.

Q15 **ADVANCED** — TCL upvar and Procedure Scope**Code Review**

```

1  proc multiply_all {arr_name factor} {
2      upvar $arr_name arr
3      foreach key [array names arr] {
4          set arr($key) [expr $arr($key) * $factor]
5      }
6  }
7  array set prices {apples 5 oranges 3 bananas 2}
8  multiply_all prices 10
```

- a) Without `upvar`, what happens to the array inside the proc?
- b) After execution, what are the values in `prices`?

Expected Answer

- a) Tcl passes by value — a local copy is modified and the original is unchanged. `upvar` creates an alias to the caller's variable.
- b) apples=50, oranges=30, bananas=20.

Q16 **MODERATE** — File I/O for Testbench Generation*Scenario*

Write a Tcl script that reads `interface.v`, replaces `input` with `reg` and `output` with `wire`, then writes the result to `tb_signals.v`.

Expected Answer

```
1 set fh [open "interface.v" r]
2 set content [read $fh]; close $fh
3 regsub -all {\binput\b} $content "reg" content
4 regsub -all {\boutput\b} $content "wire" content
5 set fh [open "tb_signals.v" w+]
6 puts $fh $content; close $fh
```

Q17 **ADVANCED** — Timing Budget Calculation in TCL*Scenario*

Write Tcl commands that set input delay to 30% and output delay to 20% of clock period `$CLK_PER` and print both values.

Expected Answer

```
1 set in_delay [expr 0.30 * $CLK_PER]
2 set out_delay [expr 0.20 * $CLK_PER]
3 puts "Input delay:  $in_delay ns"
4 puts "Output delay: $out_delay ns"
```

Q18 **ADVANCED** — Regex Report Parsing*Scenario*

Write a Tcl script that opens `timing.rpt` and prints every line containing "slack (VIOLATED)".

Expected Answer

```
1 set fh [open "timing.rpt" r]
2 while {[gets $fh line] >= 0} {
3     if {[regexp {slack \s(VIOLATED\s)} $line]} {
4         puts $line
5     }
6 }
7 close $fh
```

Q19 **MODERATE** — TCL List Operations*Fill-in-the-Blank*

Given set `clks {CLK_A CLK_B CLK_C CLK_D}`:

Task	Command
Get 3rd element	(1)
Remove CLK_B	(2)
Add CLK_E at end	(3)
Find index of CLK_C	(4)
Get length	(5)

Expected Answer

(1) `lindex $clks 2`; (2) `set clks [lreplace $clks 1 1]`; (3) `lappend clks CLK_E`; (4) `lsearch $clks CLK_C`; (5) `llength $clks`.

Q20 **ADVANCED** — Conditional Port Existence Check*Scenario*

Write a Tcl snippet in DC that applies `set_false_path` from `RST_N` only if the port exists.

Expected Answer

```
1  if {[sizeof_collection [get_ports -quiet RST_N]] > 0} {
2      set_false_path -from [get_ports RST_N]
3      echo "Applied false path on RST_N"
4  }
```

3 SpyGlass Lint & CDC Static Verification (10 Questions)

Q21 **MODERATE** — Lint Policy Hierarchy*Conceptual*

List the 5 core SpyGlass Lint policy groups and briefly describe what each catches.

Expected Answer

1. **lint** — syntax/semantic, type mismatches.
2. **morelint** — advanced style, unused signals (UnloadedOutTerm-ML).
3. **simulation** — Sim/Synth mismatches: W415 (blocking in seq), W18 (incomplete sensitivity list).
4. **latch** — inferred latches (InferLatch / W188).
5. **erc** — electrical rules: combinational loops (W120), multi-driven nets (W422), floating inputs (W240).

Q22 **ADVANCED** — Multi-Driven Net Hazard*Scenario*

```

1 module top (input a, b, output out);
2     assign out = a;
3     assign out = b;
4 endmodule

```

- Which SpyGlass ERC rule fires?
- What happens in simulation when a=1, b=0?
- What physical hardware hazard does this represent?

Expected Answer

- W422** — multi-driven net.
- Simulation result is non-deterministic or X; the simulator resolves conflicting drivers by strength or unknown.
- Physical short-circuit between two driver outputs — can destroy gates and cause hot carrier injection.

Q23 **ADVANCED** — Waiver File Strategy*Scenario*

An output port `STATUS_FLAGS[7:0]` is intentionally unconnected (optional debug port). Write the SpyGlass waiver entry.

Expected Answer

```

1 waive -rule "UnloadedOutTerm-ML" \
2     -module "ALU_TOP" \
3     -port "STATUS_FLAGS" \
4     -comment "Debug-only port. Intentionally unconnected
              in production."

```

Q24 **ADVANCED** — CDC Goal Progression*Conceptual*

Explain the 4-goal SpyGlass CDC progression and why the order is mandatory:

`cdc_setup_check` → `clock_reset_integrity` → `cdc_verify_struct` → `cdc_verify`

Expected Answer

- cdc_setup_check** — validates SGDC/SDC constraints. Without valid constraints, all subsequent goals produce unreliable results.
- clock_reset_integrity** — checks for glitchy clocks, floating reset nets.
- cdc_verify_struct** — detects structurally unsynchronised crossings (`Ac_unsync01/02`) and reconvergence (`Ac_conv01`).
- cdc_verify** — functional level: data stability, Gray code correctness, FIFO

protocol.
Each stage depends on the clean output of the previous stage.

Q25 **ADVANCED** — Reconvergence Hazard (Ac_conv01)*Scenario*

```
CLKA -> Flop A -> 2-FF Sync -> Dest Logic --+
                                           +--> AND Gate
                                           -> Output
CLKA -> Flop B -> 2-FF Sync -> Dest Logic --+
```

- Why does SpyGlass flag Ac_conv01 even though both signals are individually synchronised?
- What is the correct fix?

Expected Answer

- Each 2-FF synchroniser can resolve in either 1 or 2 cycles independently. If A resolves in 1 cycle and B resolves in 2 cycles, their AND gate sees data from different time-steps — a functional race on the combined output.
- Combine them into a **single** synchronised control token: route one signal through a single 2-FF and gate the other based on that single synchronised enable.

Q26 **MODERATE** — SGDC Clock Domain Binding*Code Review*

Write an SGDC snippet that declares `sys_clk` (100 MHz) and `usb_clk` (60 MHz) as unrelated asynchronous domains, and `rst_n` as an asynchronous active-low reset.

Expected Answer

```
1 current_design top
2 clock -name "top.sys_clk" -domain d_sys -period 10
  -waveform {0 5}
3 clock -name "top.usb_clk" -domain d_usb -period 16.67
  -waveform {0 8.33}
4 reset -name "top.rst_n" -value 0 -async
5 cdc_attribute -unrelated {top.sys_clk top.usb_clk}
```

Q27 **ADVANCED** — Glitch on Synchroniser Input (Ac_glitch01)*Scenario*

```
1 // WRONG: combinational signal fed directly to
  synchroniser
2 assign crossing_signal = (state == RUN) & enable;
3 // Then passed directly to 2-FF synchroniser in CLKB
  domain
```

Why is this a CDC violation and what is the correct coding pattern?

Expected Answer

Combinational logic can produce glitches (hazard spikes shorter than a clock period) — the synchroniser can accidentally sample a glitch as a valid transition, corrupting the destination logic.

Correct pattern: register `crossing_signal` in the source clock domain first, then feed the *registered* output into the 2-FF synchroniser. Only stable, registered signals should cross domain boundaries.

Q28 **MODERATE** — Quasi-Static Signal Definition

Conceptual

- a) What is a quasi-static signal in SpyGlass CDC terminology?
- b) When is it safe to waive a CDC violation for a quasi-static signal?
- c) Give a real-world example.

Expected Answer

- a) A signal changed only during system initialisation or controlled software configuration phases — never toggles during normal operation.
- b) Safe to waive when software guarantees the signal is stable before any destination clock domain logic reads it.
- c) Baud rate divisor register, DMA burst length configuration, mode select register.

Q29 **ADVANCED** — Clock Integrity Violation

Scenario

A design uses `assign gated_clk = clk & enable;`.

- a) Which SpyGlass CDC rule fires?
- b) What is the specific danger of using standard logic gates for clock gating?
- c) What is the correct hardware solution?

Expected Answer

- a) **Clock_glitch01**.
- b) The AND gate can produce glitches when `enable` changes while `clk` is high — the glitch propagates as a false clock edge, corrupting flip-flop data.
- c) Use an **Integrated Clock Gating (ICG)** cell: the enable is sampled by a latch on the falling edge, so `gated_clk` transitions only when `clk` is low — glitch-free.

Q30 MODERATE — SpyGlass Flow Deliverables*Fill-in-the-Blank*

Match each SpyGlass output file to its purpose:

File	Purpose
moresimple.rpt	(1)
summary.rpt	(2)
elab_summary.rpt	(3)
.awl	(4)

Expected Answer

- (1) Filtered detailed violation list with file/line references;
- (2) High-level severity count (Errors / Warnings / Infos);
- (3) Design elaboration log — port connectivity, hierarchy resolution;
- (4) Waiver file — suppresses known-benign violations with documented justification.

4 ASIC Synthesis — Synopsys Design Compiler (12 Questions)

Q31 MODERATE — PVT Library Corners*Conceptual*

- a) What do SS, TT, FF stand for in standard cell library naming?
- b) Which corner is used for **setup** analysis and why?
- c) Which corner is used for **hold** analysis and why?

Expected Answer

- a) **SS** = Slow-Slow (worst process + lowest voltage + highest temperature); **TT** = Typical-Typical; **FF** = Fast-Fast (best case).
- b) **SS** for setup — cells are slowest, giving worst-case (maximum) propagation delays — hardest setup scenario.
- c) **FF** for hold — cells are fastest, giving minimum propagation delays — data may race to corrupt the same-edge capture.

Q32 ADVANCED — link vs. elaborate vs. check_design*Conceptual*

Describe what each DC command does and in what order they must be called: analyze, elaborate, link, check_design.

Expected Answer

1. **analyze -format verilog** — parses and syntax-checks RTL into DC internal format.
2. **elaborate** — instantiates the design hierarchy, resolves parameters and generics.
3. **link** — resolves all module references against the library/design database; fails if unresolved black boxes exist.
4. **check_design** — reports structural issues (unconnected pins, multi-driven nets, missing connections). Must be clean (0 errors) before applying constraints.

Q33 ADVANCED — Path Grouping Strategy*Conceptual + Scenario*

- a) What is the risk of not using path groups?
- b) Write the 3 standard path group commands.
- c) How does path grouping improve QoR?

Expected Answer

- a) DC targets the single globally worst slack path — may fix I/O paths while ignoring critical Reg-to-Reg paths, or vice versa.

```

1  group_path -name INREG -from [all_inputs]
2  group_path -name REGOUT -to [all_outputs]
3  group_path -name INOUT -from [all_inputs] -to [
    all_outputs]

```

- b) Each group receives an independent optimisation budget — DC simultaneously targets all timing categories, producing balanced WNS across all path types.

Q34 ADVANCED — Multicycle Path Constraints*Mathematical + Scenario*

A divider uses 2 clock cycles (100 MHz clock, period = 10 ns).

- a) Write the SDC commands for setup and hold MCP constraints.
- b) What does the **-end** qualifier do in the hold constraint?
- c) What happens if MCP setup = 2 but the hold constraint is omitted?

Expected Answer

```

1  set_multicycle_path -setup 2 -from [get_clocks CLK] \
2                               -to [get_cells U_DIV/
                                   result*]
3  set_multicycle_path -hold 1 -from [get_clocks CLK] \
4                               -to [get_cells U_DIV/
                                   result*] -end

```

- b) **-end** moves the hold check reference edge at the **capture** side back by $N - 1$ cycles, preventing over-tightening.

- c) The tool defaults hold to check at cycle 0 against the setup edge of cycle 2 — an impossibly tight hold check causing unnecessary buffer insertion and area/power bloat.

Q35 **ADVANCED** — Clock Gating for Power Reduction *Conceptual + Code*

- a) What is Automatic Clock Gating (ICG) in Design Compiler?
 b) Write the DC command for ICG with minimum bit-width of 4.
 c) Why is `set_dont_touch_network` applied to clock nets before `compile_ultra`?

Expected Answer

- a) DC identifies registers with a common clock enable and inserts an ICG cell that gates the clock — saving dynamic power when registers hold stable values.
 b) `set_clock_gating_style -minimum_bitwidth 4 -positive_edge_logic {integrated}`
 c) DC would otherwise insert its own clock buffers during optimisation, unbalancing the clock tree. CTS in PnR handles clock buffering with full physical awareness.

Q36 **MODERATE** — `set_fix_multiple_port_nets` *Conceptual*

- a) What problem does `set_fix_multiple_port_nets -all -buffer_constants -feedthroughs` solve?
 b) What is a “feedthrough net” and why must it be buffered?

Expected Answer

- a) Raw Verilog netlists can contain `assign` statements for constant drivers (tied inputs) and feedthrough nets (inputs directly wired to outputs). These are valid RTL but illegal in a gate-level netlist for PnR. The command inserts buffer cells to eliminate them.
 b) A feedthrough is a primary input directly connected to a primary output with no logic. PnR requires an explicit buffer cell so the net has a driver with physical coordinates.

Q37 **ADVANCED** — Generated Clock Definition *Code Review*

```
1 create_generated_clock -name "REG_CLK" \
2   -source [get_ports CLK] \
3   -master_clock MASTER_CLK \
4   -divide_by 2 \
5   [get_pins U0_ClkDiv/o_div_clk]
```

- a) What does `-divide_by 2` mean for timing analysis?

- b) Why use `create_generated_clock` instead of `create_clock` here?
 c) What happens to Reg-to-Reg paths between `MASTER_CLK` and `REG_CLK`?

Expected Answer

- a) DC treats `REG_CLK` as having half the frequency (20 ns period if master is 10 ns).
 b) `create_clock` on an internal pin implies an independent source — DC loses the phase relationship. `create_generated_clock` maintains the parent-child relationship and propagates source latency correctly.
 c) These are CDC paths requiring `set_clock_groups -asynchronous` or `set_multicycle_path` depending on design intent.

Q38 MODERATE — Synthesis Output Deliverables

Fill-in-the-Blank

File	Full Name	Used By
<code>top.v</code> (gate-level)	(1)	(2)
<code>top.ddc</code>	(3)	(4)
<code>top.sdc</code>	(5)	(6)
<code>top.sdf</code>	(7)	(8)

Expected Answer

- (1) Verilog gate-level netlist; (2) PnR, GLS, Formality;
 (3) Synopsys Design Database binary; (4) Formality, incremental DC runs;
 (5) Synopsys Design Constraints; (6) PrimeTime STA, PnR;
 (7) Standard Delay Format; (8) GLS back-annotation in ModelSim/QuartaSim.

Q39 ADVANCED — `compile_ultra` vs. `compile`

Conceptual

What are the 3 key algorithmic differences between `compile_ultra` and `compile -map_effort high`?

Expected Answer

- Automatic hierarchy ungrouping** — `compile_ultra` flattens across module boundaries for global optimisation (better QoR, slower runtime).
- Register retiming** — moves registers across combinational logic cones.
- Advanced delay calculation** — more accurate RC models for critical path selection.

Use `compile -no_autoungroup` when: preserving hierarchy for Formality, limited runtime, or hard-macro boundaries must not be flattened.

Q40 **ADVANCED** — Wire Load Model*Conceptual*

- a) What is a Wire Load Model (WLM) and when is it used?
- b) Why do post-PnR timing results often differ from synthesis predictions?
- c) What is the most accurate timing sign-off flow?

Expected Answer

- a) A statistical model estimating wire RC based on net fanout, used in pre-layout synthesis when no physical routing exists.
- b) WLM is an average; actual routing depends on cell placement, congestion, and detours — actual RC can differ significantly.
- c) Post-PnR SPEF back-annotation into PrimeTime for parasitic-based STA.

Q41 **MODERATE** — check_design Output Interpretation*Scenario*After running `check_design`:

```
Warning: In design 'ALU_TOP', port 'STATUS[3]' is not
connected. (ELAB-311)
Error: In design 'ALU_TOP', net 'carry_internal' has
multiple drivers. (ELAB-318)
```

- a) Which warning can be safely waived and how?
- b) Which error is a showstopper and what is its fix?

Expected Answer

- a) The unconnected port warning (ELAB-311) can be waived if `STATUS[3]` is an intentional debug port left floating. Add a SpyGlass/DC waiver with a justification comment.
- b) The multi-driver error (ELAB-318) is a showstopper — two logic drivers fight on the same net. Fix: trace both drivers and remove the redundant `assign` or redesign the datapath so only one source drives `carry_internal`.

Q42 **ADVANCED** — Operating Conditions & Multi-Corner Analysis *Conceptual*

```
1 set_operating_conditions -max TSMC_SS_1P08V_125C \
2                           -min TSMC_FF_1P32V_M40C \
3                           -max_library $SSLIB \
4                           -min_library $FFLIB
```

- a) What do `-max` and `-min` operating conditions represent?
- b) What is the danger of analysing hold with SS corner delays?

Expected Answer

- a) **-max** = worst-case slow corner for setup (maximum delays); **-min** = best-case fast corner for hold (minimum delays).
- b) SS corner gives pessimistic (large) delays — hold analysis at SS shows paths that are “too slow”, artificially passing hold when they might actually race at FF corner. Must use FF for hold.

5 Static Timing Analysis — Synopsys PrimeTime (12 Questions)

Q43 MODERATE — The 4 Fundamental Timing Paths*Conceptual*

Name and describe the 4 fundamental timing path types in STA with startpoint, endpoint, and bounding SDC constraints.

Expected Answer

1. **In-to-Reg**: Primary input → D-pin of FF. Bounded by `set_input_delay` + `create_clock`.
2. **Reg-to-Reg**: Q of launch FF → D of capture FF. Bounded by `create_clock` on both.
3. **Reg-to-Out**: Q of FF → Primary output. Bounded by `create_clock` + `set_output_delay`.
4. **In-to-Out**: Primary input → Primary output (combinational). Bounded by `set_input_delay` + `set_output_delay`.

Q44 ADVANCED — Setup Slack Calculation*Mathematical*

Given: $T_{\text{period}} = 10 \text{ ns}$, $T_{\text{launch,clk}} = 0.5 \text{ ns}$, $T_{\text{capture,clk}} = 0.8 \text{ ns}$, $T_{\text{cq,max}} = 0.3 \text{ ns}$, $T_{\text{comb,max}} = 6.2 \text{ ns}$, $T_{\text{setup}} = 0.15 \text{ ns}$, $T_{\text{unc}} = 0.2 \text{ ns}$.

Calculate: (1) Data Arrival Time, (2) Data Required Time, (3) Setup Slack, (4) Violation?

Expected Answer

$$\text{DAT} = T_{\text{launch,clk}} + T_{\text{cq,max}} + T_{\text{comb,max}} = 0.5 + 0.3 + 6.2 = \mathbf{7.0 \text{ ns}}$$

$$\text{DRT} = T_{\text{period}} + T_{\text{capture,clk}} - T_{\text{setup}} - T_{\text{unc}} = 10 + 0.8 - 0.15 - 0.2 = \mathbf{10.45 \text{ ns}}$$

$$\text{Slack} = 10.45 - 7.0 = \mathbf{+3.45 \text{ ns}} \quad [\text{No violation}]$$

Q45 **ADVANCED** — Hold Slack Calculation

Mathematical

Same design, minimum path: $T_{\text{launch,clk}} = 0.5$, $T_{\text{capture,clk}} = 0.8$, $T_{\text{cq,min}} = 0.1$, $T_{\text{comb,min}} = 0.2$, $T_{\text{hold}} = 0.08$, $T_{\text{unc,hold}} = 0.05$ ns.
Calculate: (1) DAT, (2) DRT, (3) Hold Slack, (4) Violation?

Expected Answer

$$\begin{aligned} \text{DAT}_{\min} &= T_{\text{launch,clk}} + T_{\text{cq,min}} + T_{\text{comb,min}} = 0.5 + 0.1 + 0.2 = \mathbf{0.8 \text{ ns}} \\ \text{DRT}_{\text{hold}} &= T_{\text{capture,clk}} + T_{\text{hold}} + T_{\text{unc,hold}} = 0.8 + 0.08 + 0.05 = \mathbf{0.93 \text{ ns}} \\ \text{Hold Slack} &= 0.8 - 0.93 = \mathbf{-0.13 \text{ ns}} \quad [\text{Hold Violation!}] \end{aligned}$$

Q46 **ADVANCED** — Clock Skew Impact Analysis

Conceptual + Mathematical

- Define positive and negative clock skew.
- Does positive skew help or hurt setup? Show using the formula.
- Does positive skew help or hurt hold?
- Why can “useful skew” be a timing closure technique?

Expected Answer

- Positive: $T_{\text{capture,clk}} > T_{\text{launch,clk}}$ (capture clock arrives later). Negative: opposite.
- Helps setup:** $\text{DRT} = T_{\text{period}} + T_{\text{capture,clk}} - T_{\text{setup}} - T_{\text{unc}}$. Larger $T_{\text{capture,clk}}$ increases DRT $\Rightarrow$ more margin.
- Hurts hold:** $\text{DRT}_{\text{hold}} = T_{\text{capture,clk}} + T_{\text{hold}} + T_{\text{unc,hold}}$. Larger $T_{\text{capture,clk}}$ raises the minimum data must arrive.
- Intentionally delaying the capture clock on setup-critical paths borrows time without changing clock frequency.

Q47 **ADVANCED** — OCV & Derating

Conceptual

- What physical phenomenon does On-Chip Variation (OCV) model?
- For setup analysis, which path is derated late and which early? Why?
- Write the PrimeTime commands for $\pm 5\%$ derating.

Expected Answer

- PVT gradients across the die — cells in different locations see different voltage drops, temperature, and process variation.
- Setup OCV: launch path derated **late** (slower = worse for setup); capture path derated **early** (faster = less required time = also worse for setup).

```
1 set_timing_derate -early 0.95 -cell_delay
```

```
2 set_timing_derate -late 1.05 -cell_delay
```

Q48 **MODERATE** — CPPR / CRPR Explained
Conceptual

- a) What is Common Path Pessimism Removal (CPPR)?
- b) Give an example of when CPPR applies.
- c) What command enables it in PrimeTime?

Expected Answer

- a) When launch and capture paths share common clock tree buffers, those cells cannot simultaneously be fast (launch) AND slow (capture). CPPR removes this artificial double-counting of shared-path variation.
- b) Two flops clocked by the same source with only a local routing difference in their last buffer — the trunk of the clock tree is common.
- c) `set timing_remove_clock_reconvergence_pessimism true`

Q49 **ADVANCED** — Recovery & Removal Checks
Conceptual

- a) What is the **Recovery** timing check for an asynchronous reset?
- b) What is the **Removal** timing check?
- c) How do they relate to Setup and Hold checks?
- d) Why must asynchronous resets be synchronously de-asserted?

Expected Answer

- a) **Recovery**: minimum time `rst_n` must de-assert *before* the next active clock edge — analogous to Setup.
- b) **Removal**: minimum time `rst_n` must remain asserted *after* the clock edge before de-assertion — analogous to Hold.
- c) Recovery ↔ Setup; Removal ↔ Hold.
- d) If `rst_n` de-asserts asynchronously too close to the clock edge, different FFs exit reset on different cycles, breaking deterministic initialisation.

Q50 **ADVANCED** — Fixing Setup vs. Hold Violations
Scenario

Identify the violation type and correct fix for each scenario:

1. Critical path slack = −500 ps; path passes through 12 logic levels.
2. Data path minimum delay = 0.15 ns; hold requirement = 0.25 ns.
3. Fixed clock frequency; setup slack = −800 ps on a single path.

Expected Answer

1. **Setup** — upsize cells on the critical path or pipeline (insert a register).
2. **Hold** — insert delay buffers on the data path to increase minimum delay above the hold requirement.
3. **Setup at fixed frequency** — restructure logic to reduce level count, or apply useful clock skew to delay the capture flop's clock.

Q51 MODERATE — WNS vs. TNS*Conceptual*

- a) Define Worst Negative Slack (WNS) and Total Negative Slack (TNS).
- b) A design has $WNS = -200$ ps and $TNS = -15,000$ ps. What does this indicate?
- c) Which metric is the hard requirement for tape-out sign-off?

Expected Answer

- a) **WNS** = slack of the single most violating path (most negative). **TNS** = sum of all negative slacks across all violating endpoints.
- b) Many paths violating (large TNS) but no single catastrophic outlier (small WNS) — suggests systematic delay issues, not one critical path.
- c) **WNS** must be ≥ 0 at sign-off. A single negative slack means the chip will malfunction.

Q52 ADVANCED — False Path vs. Multicycle Path*Scenario*

Choose the correct SDC exception for each situation:

1. Asynchronous boundary between USB controller and CPU.
2. Floating-point divider architected for 4 cycles.
3. Scan shift path between SI and SO during test mode.
4. Configuration register written only at boot.

Expected Answer

1. `set_clock_groups -asynchronous` or `set_false_path`.
2. `set_multicycle_path -setup 4 + set_multicycle_path -hold 3 -end`.
3. `set_false_path -through [get_pins */SE]`.
4. `set_false_path` — quasi-static, never sampled in functional timing.

Q53 MODERATE — $F_{\max}$ Formula*Mathematical*

Derive the expression for $F_{\max}$ in terms of T_{cq} , T_{comb} , T_{setup} , T_{unc} , and T_{skew} .

Expected Answer

$$T_{\text{period,min}} = T_{\text{cq,max}} + T_{\text{comb,max}} + T_{\text{setup}} + T_{\text{unc}} - T_{\text{skew}}$$

$$F_{\text{max}} = \frac{1}{T_{\text{period,min}}}$$

Positive skew reduces $T_{\text{period,min}}$, increasing F_{max} .

Q54 **ADVANCED** — Input/Output Delay Modelling

Scenario

External FF: $T_{\text{co}} = 2 \text{ ns}$, board trace delay = 0.5 ns , board clock skew = 1 ns , chip period = 10 ns .

- Calculate `set_input_delay` value.
- Write the SDC command.

Expected Answer

- Input delay = $T_{\text{co}} + \text{trace} + \text{skew} = 2 + 0.5 + 1 = \mathbf{3.5 \text{ ns}}$.
- `set_input_delay 3.5 -max -clock MASTER_CLK [all_inputs]`

6 Clock Domain Crossing (CDC) Design (10 Questions)

Q55 **MODERATE** — Metastability Fundamentals

Conceptual

- What is metastability and at what voltage level does it occur?
- Can it be eliminated entirely? Why or why not?
- What design technique makes metastability **safe** for reliability?

Expected Answer

- An intermediate voltage state between logic 0 and 1 where internal feedback transistors of a flip-flop are in unstable equilibrium.
- No — it is a fundamental physical property of bistable circuits when setup/hold is violated. Any async crossing can theoretically metastabilise.
- Multi-stage synchronisers add resolution time exponentially increasing MTBF, making the probability of failure negligible.

Q56 **ADVANCED** — MTBF Formula Analysis

Mathematical

Given: $\tau = 0.2 \text{ ns}$, $T_0 = 1 \text{ ns}$, $f_{\text{clk}} = 200 \text{ MHz}$, $f_{\text{data}} = 10 \text{ MHz}$, $T_{\text{clk}} = 5 \text{ ns}$, $T_{\text{setup}} = 0.15 \text{ ns}$, $T_{\text{co}} = 0.3 \text{ ns}$.

- Calculate T_{res} for a 2-FF synchroniser.
- Calculate MTBF.
- Why does a third FF dramatically improve MTBF?

Expected Answer

$$\text{MTBF} = \frac{e^{T_{\text{res}}/\tau}}{T_0 \cdot f_{\text{clk}} \cdot f_{\text{data}}}$$

- a) $T_{\text{res}} = (T_{\text{clk}} - T_{\text{setup}} - T_{\text{co}}) + T_{\text{clk}} = (5 - 0.15 - 0.3) + 5 = \mathbf{9.55 \text{ ns}}$
- b) $\text{MTBF} = e^{9.55/0.2} / (10^{-9} \times 200 \times 10^6 \times 10 \times 10^6) \approx \text{billions of years.}$
- c) Each additional FF adds T_{clk} to T_{res} . The exponential $e^{T_{\text{res}}/\tau}$ means a 5 ns addition multiplies MTBF by $e^{25} \approx 7 \times 10^{10}$.

Q57 **ADVANCED** — Gray Code in Async FIFO *Mathematical + Conceptual*

- a) Convert binary 0111 (7) to Gray code.
- b) Convert binary 1000 (8) to Gray code.
- c) Why is Gray code essential for async FIFO pointer synchronisation?
- d) Write the Verilog expression for binary-to-Gray conversion.

Expected Answer

The formula is: $\text{Gray} = \text{Binary} \oplus (\text{Binary} \gg 1)$

- a) $0111 \oplus 0011 = 0100$
- b) $1000 \oplus 0100 = 1100$
- c) Binary 7→8 changes 4 bits simultaneously; the synchroniser might capture any of 16 erroneous combinations. Gray code changes exactly 1 bit — the captured value is always either the old or new pointer (both valid FIFO positions).
- d) `assign gray = binary ^ (binary » 1);`

Q58 **ADVANCED** — Async FIFO Full/Empty Conditions *Mathematical*

For $N = 3$ (depth=8, 4-bit Gray pointers including wrap bit):

- a) Write the **empty** condition in terms of Gray-coded pointers.
- b) Write the **full** condition.
- c) Why is full checked in the write domain and empty in the read domain?

Expected Answer

- a) **Empty** (read domain): `rp_ptr_gray == wp_ptr_gray_sync`
- b) **Full** (write domain): `wp_ptr_gray[N:N-1] == ~rp_ptr_gray_sync[N:N-1]`
AND `wp_ptr_gray[N-2:0] == rp_ptr_gray_sync[N-2:0]`
- c) Full controls the write side (prevents overflow) — must use write-domain signals. Empty controls the read side (prevents underflow) — must use read-domain signals.

Q59 **ADVANCED** — FIFO Depth Calculation*Mathematical*

Write at 200 MHz (1 word/clock), read at 80 MHz (1 word/clock), burst of 120 words. Calculate: (1) Write time, (2) Words read, (3) Min FIFO depth, (4) Actual depth (power of 2).

Expected Answer

1. Write time = $120 \times 5 \text{ ns} = \mathbf{600 \text{ ns}}$
2. Words read = $600 \text{ ns} / 12.5 \text{ ns} = \mathbf{48}$
3. Min depth = $120 - 48 = \mathbf{72}$ entries
4. Actual depth = **128** (next power of 2 ≥ 72)

Q60 **MODERATE** — Reset Synchroniser Design*Conceptual + Code*

- a) Why must an async reset be asserted asynchronously but de-asserted synchronously?
- b) Describe the circuit topology of a reset synchroniser.
- c) What is the role of VCC at the D-input of the first flip-flop?

Expected Answer

- a) Async assertion: clock may not be running (power-on) — must reset immediately. Sync de-assertion: prevents different FFs from exiting reset on different cycles.
- b) $VCC \rightarrow D$ of FF1; $Q1 \rightarrow D$ of FF2; $Q2 = \text{sync_rst_n}$. Async clear (rst_n) connected to both FFs.
- c) VCC ensures that when reset releases, the FFs load logic '1' on successive clock edges, cleanly propagating the de-assertion.

Q61 **MODERATE** — Why Multiple 2-FF Synchronisers Cannot Handle Multi-Bit Data*Conceptual*

Explain exactly why placing separate 2-FF synchronisers on each bit of a 4-bit bus is incorrect.

Expected Answer

Each FF can resolve metastability in either 1 or 2 destination clock cycles independently. At any crossing, bit[0] might resolve in 1 cycle while bit[3] resolves in 2 cycles — the destination domain sees a transient vector where some bits are updated and others are not. This coherence failure produces an erroneous intermediate value that never existed in the source domain, causing functional corruption.

Q62 **ADVANCED** — Handshake Synchroniser Protocol Diagram*Conceptual +*

Describe the 4-phase handshake CDC protocol:

Phase	Action	Signal State
1	(1)	(2)
2	(3)	(4)
3	(5)	(6)
4	(7)	(8)

What is the main limitation vs. Async FIFO?

Expected Answer

- (1) Source asserts REQ, puts data on bus; (2) REQ=1, ACK=0;
 (3) Destination samples data, asserts ACK; (4) REQ=1, ACK=1;
 (5) Source sees sync ACK, de-asserts REQ; (6) REQ=0, ACK=1;
 (7) Destination de-asserts ACK; (8) REQ=0, ACK=0.

Limitation: max throughput = 1 transfer per 4 synchroniser latencies (8–12 cycles)
 — far lower than Async FIFO streaming.

Q63 **ADVANCED** — Ac_unsync02 Deep Dive*Scenario*

A 32-bit bus crosses from CLKA to CLKB directly. SpyGlass reports Ac_unsync02.

- a) Is inserting 32 parallel 2-FF synchronisers the correct fix?
 b) List 3 architecturally correct solutions with trade-offs.

Expected Answer

- a) **No** — parallel synchronisers cause bus coherence failure (see Q61).
 b) Correct solutions:
- **Async FIFO** — highest throughput, medium hardware cost, ideal for streaming.
 - **MUX-Recirculation (DATA_SYNC)** — source holds data stable, single-bit REQ synchronises load strobe — low cost but source must hold data.
 - **Handshake controller** — general-purpose, low throughput, suitable for infrequent transfers.

Q64 **MODERATE** — CDC Verification: Simulation vs. Static*Conceptual*

- a) Why is simulation alone insufficient for CDC verification?
 b) What does SpyGlass CDC provide that simulation cannot?

Expected Answer

- a) Simulation is vector-dependent. Metastability events depend on exact timing between clock edges, which varies across PVT corners. No simulation can exhaustively cover all timing scenarios.
- b) SpyGlass performs exhaustive static structural analysis — it mathematically checks all crossing paths regardless of stimulus, guaranteeing coverage of every possible metastability scenario.

7 Power Analysis & Clock Gating (5 Questions)**Q65 ADVANCED — Power Components****Mathematical + Conceptual**

- a) Write the full power equation and define each component.
- b) Clock tree: 200 MHz, 50 pF total load, 1.2 V supply. Calculate switching power.
- c) Which component dominates at high frequency? At idle?

Expected Answer

- a) $P_{total} = P_{internal} + P_{switching} + P_{leakage}$, where $P_{switching} = \frac{1}{2}CV_{DD}^2f\alpha$ and $P_{leakage} = V_{DD} \cdot I_{leakage}$.
- b) $P = \frac{1}{2} \times 50 \times 10^{-12} \times 1.44 \times 200 \times 10^6 \times 1 = 7.2 \text{ mW}$
- c) **High frequency:** dynamic power ($P_{internal} + P_{switching}$). **Idle:** leakage — increasingly critical in advanced process nodes.

Q66 ADVANCED — VCD-Based Dynamic Power**Conceptual**

- a) What does a Value Change Dump (VCD) file capture?
- b) How does PrimeTime-PX use the VCD?
- c) What is the risk of using a VCD from an untypical workload?

Expected Answer

- a) Time-stamped event-driven records of every signal transition — digital state changes and exact simulation timestamps.
- b) PT-PX extracts actual switching activity (α toggle rates) per net, multiplies by capacitance and V^2f to compute accurate dynamic power.
- c) The power report reflects only the simulated scenario. Worst-case power requires maximum-activity VCDs from stress tests.

Q67 MODERATE — Clock Gating Efficiency*Conceptual*

- a) 10,000 FFs total; 7,000 gated with average activity 0.3. How much clock power is saved?
- b) What is the minimum bit-width threshold for ICG and why?

Expected Answer

- a) Savings = $7000 \times (1 - 0.3) = 4900$ FF-clock-loads = **49%** reduction in clock-related dynamic power.
- b) Minimum 4–8 bits. Below this, the area and power of the ICG cell (latch + AND gate) exceeds the power saved. Command: `set_clock_gating_style -minimum_bitwidth 4`.

Q68 ADVANCED — SDF for GLS Power*Conceptual*

Why does GLS power analysis use SDF back-annotation while RTL power analysis uses nominal estimates?

Expected Answer

RTL power uses generic WLM and estimated activity — ballpark accuracy only. SDF provides actual cell delays and parasitic RC from physical routing. With SDF in GLS, **glitch power** (false transitions in combinational logic before steady state) is captured accurately. RTL simulation has zero propagation delay and cannot model glitches.

Q69 MODERATE — SAIF vs. VCD vs. FSDB*Fill-in-the-Blank*

Format	Full Name	Content	Best Used For
VCD	(1)	(2)	(3)
SAIF	(4)	(5)	(6)
FSDB	(7)	(8)	(9)

Expected Answer

- (1) Value Change Dump; (2) Event-based waveform with timestamps; (3) PT-PX dynamic power, waveform debug;
 (4) Switching Activity Interchange Format; (5) Statistical toggle rates / duty cycles;
 (6) Averaged power analysis, early estimates;
 (7) Fast Signal DataBase; (8) Compressed binary waveform + switching activity; (9) Synopsys VCS / Verdi / PT-PX high-speed.

8 Formal Verification — Logic Equivalence Checking (8 Questions)

Q70 **MODERATE** — LEC vs. Simulation vs. STA

Conceptual

Compare the three verification methods:

Method	Covers	Cannot Catch
Simulation	(1)	(2)
STA	(3)	(4)
LEC (Formality)	(5)	(6)

Expected Answer

- (1) Functional behaviour for applied vectors; (2) Bugs in uncovered state space, timing.
 (3) All timing paths (100% coverage); (4) Functional logic correctness, dynamic hazards.
 (5) Logical equivalence across 100% state space; (6) Does not verify the design is “correct” — only that two representations are identical.

Q71 **ADVANCED** — SVF (Setup Verification Format) — Why It Is Critical

Conceptual

- Explain why FSM re-encoding breaks Formality without SVF.
- Explain why register retiming breaks Formality without SVF.
- How is the SVF generated and consumed?

Expected Answer

- DC may change state encoding (e.g., Binary → One-Hot) — register names, bit-widths, and state values all change. Formality cannot match retimed state bits to original RTL states without SVF guidance.
- Retiming moves registers across combinational logic boundaries. A register at a module output in RTL may now be at a gate input in the netlist. Without SVF, Formality cannot find the matching compare point.
- Generated in DC:** `set_svf <name>.svf before compile`; disable with `set_svf -off`. **Consumed in fm_shell:** `set_svf <name>.svf before read_verilog`.

Q72 **ADVANCED** — Compare Points & Logic Cones

Conceptual

- What are Formality Compare Points? List the 3 types.
- What happens if a compare point in Imp has no match in Ref?

c) What does an “ABORTED” compare point mean?

Expected Answer

- a) Points where Formality cuts the circuit into combinational cones: (1) Primary output ports; (2) D-inputs of all FFs and latches; (3) Inputs to black-box modules.
- b) Listed in `unmatched_points.rpt` — may indicate unintended register removal by synthesis. Must be reviewed carefully.
- c) The SAT solver ran out of memory or time. Equivalence is neither proven nor disproven — requires debugging (cone simplification, additional SVF).

Q73 **MODERATE** — Handling Post-DFT Formal Verification *Code Review*

Formality fails with hundreds of mismatches after DFT insertion. Write the TCL commands to configure Formality for post-DFT verification.

Expected Answer

```

1  set_dont_verify_points -type port Ref:/WORK/*/SI -quiet
2  set_dont_verify_points -type port Imp:/WORK/*/SI -quiet
3  set_dont_verify_points -type port Ref:/WORK/*/SO -quiet
4  set_dont_verify_points -type port Imp:/WORK/*/SO -quiet
5  set_constant Ref:/WORK/*/SE 0 -quiet
6  set_constant Imp:/WORK/*/SE 0 -quiet
7  set_constant Ref:/WORK/*/test_mode 0 -quiet
8  set_constant Imp:/WORK/*/test_mode 0 -quiet

```

Q74 **ADVANCED** — Formality vs. Simulation: Exhaustiveness *Conceptual*

A design has 32 state registers. How many input combinations must simulation cover exhaustively vs. what Formality proves?

Expected Answer

Simulation: $2^{32} \times (\text{input combinations per state}) > 4 \times 10^9$ state combinations $\times$ input space — computationally impossible.

Formality: Uses BDD (Binary Decision Diagrams) and SAT solvers to reason about ALL possible states simultaneously — proves equivalence across the entire state space in minutes without enumeration.

Q75 **ADVANCED** — Diagnosing Formal Failures*Scenario*

```

FAILED: 3 compare points
- reg_out[7]: FAILED
- reg_out[6]: FAILED
- primary_output_STATUS: FAILED

```

- What command do you run to investigate?
- What does `analyze_points -failing` show?
- List 3 common root causes for Formality failures.

Expected Answer

- `diagnose; analyze_points -failing`
- Shows the counter-example input vector that distinguishes the two implementations.
- (1) Missing SVF — DC optimisation changed structure without guidance; (2) DFT test mode active (`SE=1` during verification); (3) Stale netlist vs. new RTL — synthesis not re-run after RTL changes.

Q76 **MODERATE** — Ref vs. Imp Containers*Scenario*

Verifying a post-PnR netlist against original RTL. Which design goes in **Ref** and **Imp**? What files must be loaded into each?

Expected Answer

Ref: RTL (all source `.v` files via `read_verilog`) + standard cell `.db` libraries.
Imp: Post-PnR netlist (`top_pnr.v`) + same `.db` libraries. SVF loaded globally before reading any design.

Q77 **ADVANCED** — Incremental LEC Runtime Optimisation*Conceptual*

Formality takes 12 hours on a 500K-gate design. What 3 techniques improve runtime without sacrificing coverage?

Expected Answer

- Hierarchical verification** — verify sub-blocks independently with defined boundaries, then integrate at the top.
- Restrict scope** with `set_dont_verify_points` for known-stable blocks that have not changed.
- Comprehensive SVF** — ensures Formality maps points efficiently without wasting time on unmatched topology exploration.

9 Design for Testability (DFT) & Scan Insertion (10 Questions)

Q78 **MODERATE** — Why DFT?

Conceptual

Distinguish between **functional verification** and **manufacturing test**. Why can a chip pass all RTL simulations but still fail manufacturing test?

Expected Answer

Functional verification confirms logical behaviour matches the specification. Manufacturing test detects physical silicon defects (metal shorts, opens, via voids, gate oxide defects) from CMOS fabrication. A chip can be logically perfect but have a metal short from a lithography defect — simulation cannot model physical process variation. ATPG generates patterns targeting physical fault models (stuck-at-0/1, transition faults).

Q79 **ADVANCED** — Muxed-D Scan Flip-Flop Operation *Conceptual + Code*

- Describe the internal architecture of a Muxed-D SDFF.
- State of SE during: normal operation, scan shift, capture?
- Write a Verilog model of a Muxed-D SDFF.

Expected Answer

- 2:1 MUX → D-FF: Functional_D (sel=0) and Scan_In (sel=1), controlled by SE.
- Normal: SE=0; Shift: SE=1; Capture: SE=0 (one clock pulse).

```
1 module SDFF (input CLK, SE, D, SI, output reg Q);
2     always @(posedge CLK)
3         Q <= SE ? SI : D;
4 endmodule
```

Q80 **ADVANCED** — DFT DRC: Clock Controllability (D1)

Scenario

An internal clock divider generates $\text{div_clk} = \text{clk}/4$. During scan shift, the tester needs full-speed clock.

- Which DFT DRC rule fires?
- Describe the fix.

Expected Answer

- D1: Clock Controllability violation.**
- Insert a `test_mode`-controlled 2:1 MUX: `gated_clk = test_mode ?`

scan_clk : div_clk. When test_mode=1, scan_clk bypasses the divider directly.

```
1 set_dft_signal -port scan_clk -type ScanClock \
2               -view existing_dft -timing {30 60}
```

Q81 **ADVANCED** — DFT DRC: Reset Controllability (D2)

Scenario

During scan shift, the asynchronous reset is active, corrupting the scan chain.

- Which DFT DRC rule fires?
- Write the gate-level fix.

Expected Answer

- D2: Reset Controllability violation.**
- Gate reset with test_mode: `actual_rst_n = rst_n | test_mode;` — when test_mode=1, reset is held inactive (logic 1 for active-low), allowing free scan shifting.

```
1 set_dft_signal -port scan_rst -type Reset \
2               -view existing_dft -active_state 0
```

Q82 **MODERATE** — Scan Chain Architecture

Conceptual

- Why is `-clock_mixing no_mix` important?
- What happens with mixed positive/negative-edge FFs in the same chain?
- Why does DFT Compiler place negative-edge FFs first?

Expected Answer

- Mixing clocks from different domains causes hold violations during scan shift — chain data races across domains on a single shift clock pulse.
- On a rising shift edge, the pos-edge flop captures new data while the neg-edge flop still holds old data — the old data may be overwritten before being shifted out.
- Neg-edge FFs placed first capture SI on the rising edge and pass to pos-edge FFs on the falling edge — avoiding data overlap.

Q83 **ADVANCED** — ATPG Coverage

Conceptual

- What is stuck-at fault coverage and why does it matter?
- What is the production tape-out target?
- What are “untestable” faults and how are they classified?

Expected Answer

- a) Percentage of all modelled stuck-at-0/stuck-at-1 faults detectable by ATPG patterns. Higher coverage = higher confidence that no defective chip escapes.
- b) $\geq 98\%$ stuck-at coverage (often 99%+).
- c) Nodes that cannot be controlled or observed via any input/scan access. Classified as: **Redundant** (logically masked), **Unused** (dead code), **Tied** (constant by design).

Q84 **ADVANCED** — Post-DFT Deliverables to TetraMAX *Fill-in-the-Blank*

File	Purpose in ATPG
top_dft.v	(1)
top_dft.spf	(2)
top_dft.sdc	(3)
scan_paths.rpt	(4)

Expected Answer

- (1) Gate-level netlist — TetraMAX uses structural connectivity to model faults and generate patterns.
- (2) STIL Protocol File — defines scan chain connectivity (SI/SO, SE, scan clock waveforms).
- (3) Timing constraints — ensures ATPG patterns respect scan clock periods and hold times.
- (4) Scan path report — audit document verifying chain integrity before ATPG.

Q85 **MODERATE** — `compile -scan` vs. `insert_dft` *Conceptual*

What is the functional difference between these two DC-DFT commands?

Expected Answer

compile -scan: replaces standard D flip-flops with *unstitched* scan flip-flops — each Sdff has SI and SO pins floating. No chain connections made yet.
insert_dft: physically stitches unconnected scan ports into ordered chains:

$$\text{SI} \rightarrow \text{Sdff}_1 \rightarrow \text{Sdff}_2 \rightarrow \dots \rightarrow \text{Sdff}_n \rightarrow \text{SO}$$

After `insert_dft`, a complete, functional scan path exists.

Q86 **ADVANCED** — DFT's Impact on Formal Verification*Conceptual*

After DFT insertion, Formality between pre-DFT RTL and post-DFT netlist fails. Name the 5 specific changes DFT introduces that Formality must account for.

Expected Answer

1. **New ports** added: SI, SO, SE, scan_clk, scan_rst, test_mode — absent in RTL.
2. **MUX at every FF D-input** — changes logic topology throughout.
3. **Scan chain routing** changes FF interconnect.
4. **test_mode MUX bypasses** clock dividers — creates alternative clock paths.
5. **Reset gating** with test_mode. Fix: set_dont_verify_points on SI/SO; set_constant SE=0, test_mode=0 to force functional mode.

10 Gate-Level Simulation (GLS) & Back-Annotation (5 Questions)

Q87 **MODERATE** — Why GLS After STA?*Conceptual*

STA proves all timing paths clean. LEC proves netlist matches RTL. List at least 4 additional failures that GLS can catch.

Expected Answer

1. **Async reset recovery/removal violations** — real sequential impact on downstream logic after metastable reset de-assertion.
2. **CDC functional verification** — validates handshake protocols and FIFO behaviour with real delays.
3. **Glitch detection** — combinational hazards from unequal path delays not visible in zero-delay RTL simulation.
4. **Multi-cycle / false path validation** — confirms logic completes in allocated cycles under real delays.
5. **X-state propagation** — ensures all FFs initialise correctly on reset.

Q88 **ADVANCED** — SDF Annotation Modes*Scenario*

Mode	vsim Flag	Purpose	PVT Corner
Zero-delay	+notimingchecks	(1)	(2)
SDF Max	-sdfmax	(3)	(4)
SDF Min	-sdfmin	(5)	(6)

Expected Answer

- (1) Pure functional verification — confirms logic correct without timing; (2) N/A (no delays);
 (3) Setup-time worst-case: maximum delays show worst-case data arrival; (4) SS 1.08V 125°C;
 (5) Hold-time: minimum delays reveal fastest data propagation and hold races; (6) FF 1.32V –40°C.

Q89 **ADVANCED** — X-Propagation Problem*Scenario*

In GLS with SDF max, 80% of outputs show XXXX from time 0 and never resolve.

- a) Most likely cause?
- b) Diagnostic approach?
- c) Fix?

Expected Answer

- a) FFs without asynchronous reset power up as X in gate-level models. Any gate receiving X propagates X to output — cascading X through the entire design.
- b) Check if async reset pulse is applied at $t = 0$ in the testbench; verify SDF load order; use `+acc` to inspect individual FF states.
- c) Apply a clean reset pulse for ≥ 2 clock cycles. FFs without async reset must be force-initialised via `$deposit` or simulator `force`.

Q90 **ADVANCED** — Notifier Register Mechanism*Conceptual*

GLS runs correctly for 10 μ s then starts producing X on several outputs.

- a) What is a timing notifier register?
- b) How does it relate to the observed X?
- c) What are the two strategies to handle this?

Expected Answer

- a) A Verilog `$setup/$hold` task inside a `specify` block. When a timing violation is detected, it writes X to a notifier register, forcing the FF Q to X.
- b) A legitimate setup or hold violation (often near mode transitions) triggers the notifier and propagates X downstream.
- c) (1) **Debug mode:** use `+timing_checks` and timing reports to find the violating path. (2) **Suppress during initialisation:** use `+notimingchecks` or `+no_notifier` only during reset, then re-enable.

Q91 **MODERATE** — PrimeTime-PX Power Analysis Flow*Conceptual*

Describe the 5-step PrimeTime-PX flow for dynamic power analysis after GLS.

Expected Answer

```

1  # Step 1: Enable power mode
2  set_app_var power_enable_analysis true
3  set_app_var power_analysis_mode    time_based
4
5  # Step 2: Read gate netlist
6  read_verilog "results/top_netlist.v"
7  current_design top
8  link_design
9
10 # Step 3: Read timing
11 read_sdc "results/top.sdc"
12 read_sdf "results/top.sdf"
13
14 # Step 4: Annotate switching activity
15 read_vcd -strip_path tb/dut "sim/VCD/design.vcd"
16
17 # Step 5: Calculate and report
18 update_power
19 report_power > "pt_reports/power_summary.rpt"
20 report_power -hierarchy > "pt_reports/power_hierarchy.rpt"

```

11 Physical Design — Place & Route (8 Questions)

Q92 **MODERATE** — PnR Flow Stages*Conceptual*

Name all 7 major stages of the physical design flow in order with a one-sentence description of each.

Expected Answer

1. **Design Import & MMMC Setup** — load netlist, LEF, lib, define multi-corner views.
2. **Floorplanning & Power Planning** — define die/core area, insert VDD/VSS power rings and stripes.
3. **Standard Cell Placement** — legally place all cells minimising wirelength and congestion.
4. **Clock Tree Synthesis (CTS)** — build a balanced clock buffer tree minimising skew and insertion delay.
5. **Global & Detailed Routing** — NanoRoute assigns metal tracks and vias to all nets.
6. **Chip Finishing** — filler cells, antenna fix, DRC/LVS verification.
7. **Signoff Export** — GDSII, SPEF, post-PnR netlist, SDF.

Q93 **ADVANCED** — Floorplan Utilisation & Aspect Ratio *Mathematical*

Total cell area: $15,000 \mu\text{m}^2$, die: $200 \mu\text{m} \times 150 \mu\text{m}$, core margins: $5 \mu\text{m}$ all sides.
Calculate: (1) Core area, (2) Utilisation, (3) Aspect ratio, (4) Is this reasonable?

Expected Answer

$$\text{Core area} = (200 - 10) \times (150 - 10) = 190 \times 140 = \mathbf{26,600 \mu\text{m}^2}$$

$$\text{Utilisation} = 15,000 / 26,600 = \mathbf{56.4\%}$$

$$\text{Aspect ratio} = 140 / 190 = \mathbf{0.74}$$

Reasonable: 50–70% utilisation is typical. Below 50% wastes area; above 80% causes routing congestion and CTS difficulties.

Q94 **ADVANCED** — CTS Targets *Conceptual + Scenario*

- Define clock skew, insertion delay, and transition time in the context of CTS.
- After CTS, skew = 450 ps but target is ≤ 200 ps. Give 3 reduction approaches.
- Why must hold violations be re-checked after CTS?

Expected Answer

- Skew:** max clock arrival time difference between any two FFs. **Insertion delay:** time for clock to travel from source pin to FF clock pins. **Transition (slew):** rise/fall time of clock waveform at FF pins.
- (1) Restructure clock tree topology to a more balanced H-tree. (2) Add buffering on late-arriving branches. (3) Upsize clock buffers on high-fanout nodes.
- Before CTS, clocks are ideal (zero delay). After CTS, real insertion delays exist — paths with marginal hold during synthesis may now violate because the capture FF receives its clock later.

Q95 **ADVANCED** — Power Delivery Network (PDN) *Conceptual*

- Describe the 3-tier power delivery hierarchy in an ASIC.
- What is IR drop and why is it dangerous?
- What is electromigration and how does it constrain power stripe width?

Expected Answer

- (1) **Power Rings** — thick metal rings around core carrying VDD/VSS from IO pads. (2) **Power Stripes** — wide upper-layer (M5/M6) lines distributing current across the core. (3) **Standard Cell Rails** — M1 horizontal lines in each cell row.

- b) IR drop = $V = I \times R$. Excessive drop lowers V_{DD} at cell locations $\Rightarrow$ slower cells $\Rightarrow$ setup violations and functional failure.
- c) Electromigration: electron wind physically displaces metal atoms over time, creating voids or shorts. Each layer has a maximum current density limit ($A/\mu m$); exceeding it causes reliability failure. Wider stripes reduce current density below the limit.

Q96 **ADVANCED** — Filler Cells vs. Tie Cells*Conceptual*

- a) Why are filler cells needed and what do they contain?
- b) Why can't unused gate inputs connect directly to VDD or VSS?
- c) What cells replace direct VDD/VSS connections?

Expected Answer

- a) Empty row gaps break continuous N-well/P-substrate implant layers $\Rightarrow$ DRC violations. Filler cells contain no active transistors but maintain implant continuity.
- b) Direct connections are subject to ESD events during PCB handling — the voltage spike can rupture thin gate oxides (< 5 nm in modern nodes).
- c) **Tie-Hi (TIEHIM)** and **Tie-Lo (TIELOM)** cells: contain a protected circuit with internal ESD clamps that safely output logic 1 or 0.

Q97 **MODERATE** — GDSII Tape-Out Process*Conceptual*

- a) What does “tape-out” mean in the IC design flow?
- b) What is GDSII and what does it contain?
- c) What are the two final foundry verification steps after GDSII generation?

Expected Answer

- a) Tape-out is the final delivery of the chip design to the semiconductor foundry. No further RTL or physical changes can be made. Historically referred to the magnetic tape used to carry design data.
- b) GDSII (Graphic Data System II) is a binary stream format containing all physical mask geometries — every polygon, via, text label, and layer assignment needed to expose photolithography masks.
- c) (1) **DRC** — verifies all geometries meet foundry minimum spacing, width, and density rules. (2) **LVS** — verifies the extracted layout netlist matches the gate-level schematic connectivity.

Q98 **ADVANCED** — Antenna Violation*Conceptual*

- a) What causes the antenna effect during fabrication?
- b) What is the danger to the circuit?

c) Name two methods to fix antenna violations.

Expected Answer

- a) During plasma etching, long metal routes accumulate charge from the ionised gas. When charge is large enough, it drives current through connected gate oxides before the gate's source/drain provides a discharge path.
- b) The charge can rupture the thin gate oxide (2–5 nm) permanently, causing transistor failure or V_{th} shift — parametric yield loss.
- c) (1) **Antenna diode insertion** — diode-connected transistor at gate input discharges charge through substrate. (2) **Metal jumper** — break the long route and re-route through a higher layer with an intervening via (resets the antenna ratio).

Q99 **ADVANCED** — MMMC Setup in PnR

Conceptual

- a) What does MMMC (Multi-Mode Multi-Corner) mean in physical design?
- b) Why are both `max_library` (SS) and `min_library` (FF) needed simultaneously?
- c) How does Encounter use MMMC during post-CTS timing optimisation?

Expected Answer

- a) MMMC simultaneously analyses the design across multiple operating modes (functional, test, low-power) and multiple PVT corners (SS 125°C for setup, FF –40°C for hold) in a single PnR session.
- b) Setup optimisation needs worst-case SS timing; hold needs best-case FF. Without both, fixing setup may introduce hold violations — MMMC prevents oscillating fixes.
- c) After CTS, Encounter runs all corners simultaneously. Buffers inserted for hold at FF corner are verified not to create setup issues at SS corner, and vice versa.

Q100 **ADVANCED** — Full ASIC Flow Integration

System-Level Scenario

A chip returns from the foundry with 30% failures producing wrong outputs only at 125°C and 500 MHz.

- a) What type of failure is most likely?
- b) Which ASIC flow steps may have failed to catch this?
- c) Propose a complete corrective action plan for the next tape-out.

Expected Answer

- a) **Setup timing violation** at the Slow-Slow process corner (SS + 125°C) — cells at their slowest, paths marginally passing at typical conditions violate at

worst case.

b) Possible flow gaps:

- STA run only at TT corner, not SS 125°C.
- WLM inaccurate pre-layout; actual routing RC worse.
- OCV derating not applied.
- SPEF back-annotation from PnR not used for final STA.
- Clock uncertainty margin insufficient.

c) Corrective plan:

- Re-run STA at SS 1.08V 125°C with SPEF back-annotation.
- Enable OCV derating ($\pm 5\%$ or foundry AOCV tables).
- Increase clock uncertainty margin to cover measured jitter + skew.
- Fix critical paths — upsize cells or increase pipeline depth.
- Add timing margin guard of +200–500 ps to all critical endpoints before tape-out.

Summary Statistics

Section	Topic	Questions	Difficulty
1	Verilog HDL & Digital Architecture	12	8 Adv / 4 Mod
2	TCL Scripting	8	5 Adv / 3 Mod
3	SpyGlass Lint & CDC	10	7 Adv / 3 Mod
4	ASIC Synthesis (DC)	12	9 Adv / 3 Mod
5	Static Timing Analysis (PrimeTime)	12	9 Adv / 3 Mod
6	CDC Design	10	7 Adv / 3 Mod
7	Power Analysis	5	3 Adv / 2 Mod
8	Formal Verification (Formality)	8	6 Adv / 2 Mod
9	DFT & Scan Insertion	10	7 Adv / 3 Mod
10	Gate-Level Simulation	5	4 Adv / 1 Mod
11	Physical Design (PnR)	8	6 Adv / 2 Mod
Total		100	71 Adv / 29 Mod

Interview Tips & Formula Reference

Interview Tip

For behavioural interviews: Always connect answers to diploma project experience — mention SpyGlass, Design Compiler, PrimeTime, Formality, or Cadence Encounter by name.

Key role focus areas:

- **RTL / Digital Design:** Q1–Q12, Q55–Q64 (CDC always asked).
- **Synthesis / STA:** Q31–Q54 heavily tested.
- **DFT Specialist:** Q78–Q86 are critical.
- **Physical Design:** Q92–Q100 are the foundation.
- **Verification:** Q70–Q77 (Formal), Q87–Q91 (GLS).

Formula Reference

Formula Cheat Sheet — Memorise These:

$$\text{Setup Slack} = (T_{\text{period}} + T_{\text{capture,clk}} - T_{\text{setup}} - T_{\text{unc}}) - (T_{\text{launch,clk}} + T_{\text{cq,max}} + T_{\text{comb,max}}) \quad (1)$$

$$\text{Hold Slack} = (T_{\text{launch,clk}} + T_{\text{cq,min}} + T_{\text{comb,min}}) - (T_{\text{capture,clk}} + T_{\text{hold}} + T_{\text{unc,hold}}) \quad (2)$$

$$F_{\text{max}} = \frac{1}{T_{\text{cq,max}} + T_{\text{comb,max}} + T_{\text{setup}} + T_{\text{unc}} - T_{\text{skew}}} \quad (3)$$

$$\text{Gray Code} = \text{Binary} \oplus (\text{Binary} \gg 1) \quad (4)$$

$$P_{\text{switching}} = \frac{1}{2} C V_{DD}^2 f \alpha \quad (5)$$

$$\text{MTBF} = \frac{e^{T_{\text{res}}/\tau}}{T_0 \cdot f_{\text{clk}} \cdot f_{\text{data}}} \quad (6)$$